

Statistics cookbook — Social & Behavioral Sciences

For: communication science, psychology, sociology, political science, education, public health (behavioral), and marketing / PR — fields whose work is built on **surveys, Likert scales, coded content, and designed experiments**, and whose goal is usually *inference about effects* rather than prediction.

This is a **task-first** guide: find the question you're asking, copy the recipe, read how to report it. Every snippet here is executed by `examples/cookbook_social_test.bas` (run by the test suite), so the code is known-good.

Setup

```
program analysis(args)
  load stats from "stdlib/stats.bas"      ' path relative to your program
  ' ... recipes below ...
end program
```

Run with the stdlib on the path: `GBASIC_PATH=stdlib ./gbasic analysis.bas` (after `make install`, `load stats from "stats.bas"` resolves on its own).

Data conventions

- A **variable / sample** is a flat list: `[4.1, 3.8, 4.4, ...]`.
- **Groups** (for ANOVA) are a list of lists: `[[...], [...], [...]]`.
- **Predictors** for a regression are a list of columns: `[x1, x2]` (each a list). A single predictor can be passed as `[x]`.
- A **categorical predictor** (factor) is a list of labels (strings or numbers); turn it into columns with `dummy_code` (below).
- Any function returns a **record** of results, or `unknown` on bad/degenerate input — guard with `is_unknown(...)`.

1. Describe a variable

```
mean(x)    median(x)    stdev(x)    variance(x)
quantile(sort(x), 0.25)  iqr(x)      skewness(x) kurtosis(x)
```

2. Compare two conditions

Question: does the outcome differ between two groups?

```
tt = t_test_2sample(control, treatment)  ' Welch: t_test_welch; paired: t_test_paired
d  = cohens_d(control, treatment)        ' effect size
' non-normal? use the rank test:
mw = mann_whitney(control, treatment)
```

Report: $t(df) = \text{tt.statistic}$, $p = \text{tt.p_value}$, Cohen's $d = d$. Give the group means and a CI (`confidence_interval`). Cohen's $d \approx 0.2 / 0.5 / 0.8 = \text{small} / \text{medium} / \text{large}$.

3. Compare three or more groups (one-way)

```
av = anova_oneway(groups)      ' groups = [...],[...],[...]
es = eta_squared(groups)      ' eta^2 (and omega^2)
hsd = tukey_hsd(groups)       ' which pairs differ (Tukey HSD)
```

Non-parametric alternative: `kruskal_wallis(groups)`. **Report:** $F(df1, df2) = \text{av.statistic}$, $p = \text{av.p_value}$, $\eta^2 = \text{es.eta_squared}$; then the significant Tukey pairs with their CIs.

4. Factorial & repeated-measures designs

```
aw = anova_tway(cells)        ' cells[i][j] = replicate list; {a, b, interaction,
residual}
rm = anova_repeated(data)     ' data[subject][condition]; within-subjects
fr = friedman(data)           ' non-parametric repeated measures
```

Report each effect: F , p , and for two-way the interaction line — the term communication reviewers look at first.

5. Association between two variables

```
pearson: ols(y, [x]) or correlation(x, y)
spearman(x, y)                ' rank correlation {rho, p_value, n}
kendall_tau(x, y)             ' tau-b, tie-corrected
partial_correlation(x, y, z)  ' control for z
point_biserial(binary, x)     ' one dichotomous variable
cramers_v(table)              ' two categorical (from a contingency table)
```

6. Predict a continuous outcome (regression)

```
' categorical predictor -> dummy columns (first level = reference)
d = dummy_code(group)
fit = ols(y, [x, d.columns[0], d.columns[1]])
fit.coefficients  fit.std_errors  fit.p_values  fit.r_squared  fit.adj_r_squared
ci = conf_int(fit, 0.95)

' interaction term (moderation in a regression):
fit2 = ols(y, [x, d.columns[0], interaction(x, d.columns[0])])

' heteroskedasticity? robust standard errors (HC0/HC1/HC2/HC3):
rob = ols_robust(y, [x, d.columns[0], d.columns[1]], "HC3")
```

Report: b (SE), t , p per predictor, and R^2 / adjusted R^2 . With dummies, each coefficient is the contrast vs the reference level.

7. Predict a categorical or count outcome (GLM)

Outcome	Function	Report
Binary (0/1)	<code>logistic_regression(y, xs)</code>	odds ratios + CI, pseudo-R ²
Binary (probit)	<code>probit_regression(y, xs)</code>	coefficients, marginal effects
Ordered / Likert	<code>ordinal_regression(y, xs)</code>	proportional-odds ORs, cutpoints
Nominal (>2 categories)	<code>multinomial_regression(y, xs)</code>	per-category coefficients
Counts	<code>poisson_regression(y, xs)</code>	incidence-rate ratios
Over-dispersed counts	<code>negbinom_regression(y, xs)</code>	IRRs, dispersion <code>alpha</code>

```
lg = logistic_regression(yb, [x])
orr = odds_ratios(lg, 0.95)          ' orr[j] = {odds_ratio, ci_low, ci_high}
me = marginal_effects(lg, [x])      ' average marginal effects (interpretable)
lg.pseudo_r2                        ' McFadden
```

Report (logistic): OR = `orr[1].odds_ratio`, 95% CI [`ci_low`, `ci_high`], *p*; the AME (`me.effects`) as "a one-unit increase in *x* changes $P(y=1)$ by ..."; McFadden pseudo-R². **Likert outcomes belong here** — use `ordinal_regression`, not OLS.

8. Mediation & moderation

Mediation (does *X* affect *Y* through *M*?):

```
seed(42)                                ' reproducible bootstrap CI
med = mediation(y, x, m, 5000)
med.indirect      med.boot_low  med.boot_high      ' a*b with percentile CI
med.c_direct      med.c_total   med.prop_mediated
```

Report: indirect effect $a \cdot b$ = `med.indirect`, 95% bootstrap CI [`boot_low`, `boot_high`] — **significant if the CI excludes 0** (the Preacher & Hayes standard); plus direct and total effects.

Moderation (does the $X \rightarrow Y$ effect *depend* on *W*?):

```
ss = simple_slopes(y, x, w, [mean(w) - stdev(w), mean(w), mean(w) + stdev(w)])
ss.interaction_coef  ss.interaction_p          ' the X*W term
ss.slopes[k]         ' {w, slope, se, t, p_value}
```

Report: the interaction *b* and *p*; then the simple slope of *X* at low / mean / high *W* (the classic ± 1 SD probe).

9. Measurement: reliability & agreement

Scale reliability (do these items hang together?):

```
cronbach_alpha(items)          ' items[subject][item]; {alpha, n_items, n_subjects}
```

Content analysis / intercoder agreement:

```

krippendorff_alpha(coded, "nominal")    ' coded[coder][unit], unknown = missing
cohens_kappa(coder1, coder2)            ' two coders
icc(ratings)                            ' intraclass correlation, ratings[subject][rater]

```

Report: $\alpha \geq .70$ acceptable; Krippendorff $\alpha \geq .80$ ($\geq .667$ tentative). Use Krippendorff for content analysis (handles >2 coders and missing codes); "interval" / "ordinal" levels are available for scaled codes.

10. Proportions & campaign metrics

```

prop_test_1(successes, n, p0)            ' one proportion vs a benchmark
prop_test_2(s1, n1, s2, n2)             ' two proportions
ab = ab_test({successes: 90, n: 1000}, {successes: 120, n: 1000})
ab.lift    ab.p_value    ab.significant    ' relative lift + test
funnel(steps)    cohort_retention(events)    ' conversion / retention

```

11. Design: power & sample size

Run **before** collecting data (a priori power) or to justify N to reviewers:

```

power_ttest(effect_size, n, alpha, sided)    ' two-sample t-test power
power_anova(k, n, effect_size, alpha)        ' one-way ANOVA (Cohen's f)
sample_size_ttest(effect_size, power, alpha, sided)    ' required per-group n
power_ttest_paired(effect_size, n, alpha, sided)

```

Report: "To detect $d = 0.5$ at $\alpha = .05$ with 80% power (two-tailed), we need $n = \text{sample_size_ttest}(0.5, 0.8, 0.05, 2)$ per group."

12. Reporting quick reference

You have	Recipe	Headline numbers
2 groups, continuous	<code>t_test_2sample</code> + <code>cohens_d</code>	t, p, d
3+ groups	<code>anova_oneway</code> + <code>tukey_hsd</code> + <code>eta_squared</code>	F, p, η^2 , pairs
Factor $\times$ factor	<code>anova_twoway</code>	main + interaction F/p
Two continuous vars	<code>spearman</code> / <code>kendall_tau</code>	ρ / τ, p
Continuous outcome + predictors	<code>ols</code> (+ <code>ols_robust</code>)	b , SE, p , R^2
Binary outcome	<code>logistic_regression</code> + <code>odds_ratios</code>	OR, CI, pseudo- R^2
Likert outcome	<code>ordinal_regression</code>	proportional-odds OR
Counts	<code>poisson</code> / <code>negbinom_regression</code>	IRR
$X \rightarrow M \rightarrow Y$	<code>mediation</code>	indirect + bootstrap CI
Effect depends on W	<code>simple_slopes</code>	interaction p , simple slopes
Scale / coders	<code>cronbach_alpha</code> / <code>krippendorff_alpha</code>	α
Two rates / A-B	<code>prop_test_2</code> / <code>ab_test</code>	z, p , lift
Planning N	<code>sample_size_ttest</code> / <code>power_anova</code>	required n , power

All results are verified numerically against SciPy / statsmodels (GARCH against the `arch` package). See [docs/statistics_scientist_plan.md](#) for method notes and `examples/cookbook_social_test.bas` for the runnable version of every recipe above. Predictive machine learning (train/test, cross-validation, classifiers) is a separate track, intentionally out of scope here.